

AI VIET NAM – AI COURSE 2025

LLM Series: Agentic RAG

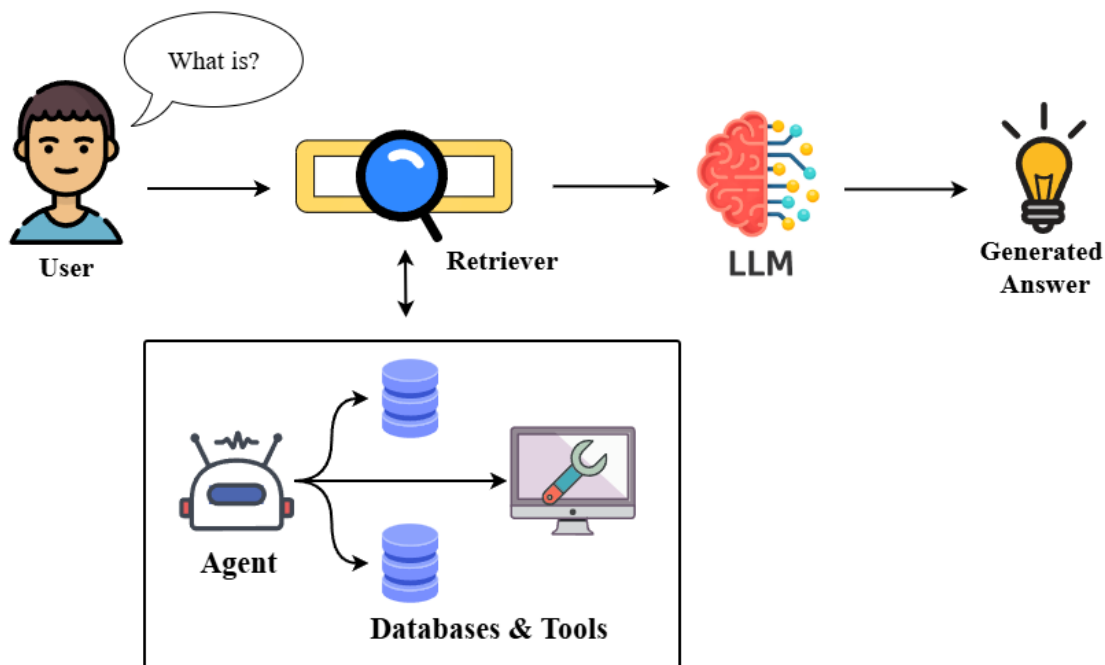
Trần Đại Nhân

Dương Nguyễn Thuận

Đinh Quang Vinh

I. Dẫn nhập

Retrieval-Augmented Generation (RAG) là một hướng tiếp cận giúp mô hình ngôn ngữ lớn khai thác tri thức bên ngoài để tạo câu trả lời chính xác hơn. Trong RAG truyền thống, hệ thống thường hoạt động theo một pipeline cố định: nhận câu hỏi, truy xuất ngữ cảnh, đưa ngữ cảnh vào large language model (LLM), rồi sinh câu trả lời. Cách làm này phù hợp với các câu hỏi đơn giản. Tuy nhiên, khi câu hỏi cần nhiều bước suy luận, cần đổi chiến lược truy xuất hoặc cần kiểm tra lại chất lượng ngữ cảnh, pipeline cố định bắt đầu trở nên kém linh hoạt.



Hình 1: Minh họa Agentic RAG.

Bài học này giới thiệu Agentic RAG trên nền tảng LangGraph và Deep Agents. Thông qua project hỏi đáp pháp luật tiếng Việt, người học sẽ thấy cách hệ thống lựa chọn chiến lược retrieval phù hợp, truy xuất ngữ cảnh và sinh câu trả lời có dẫn nguồn.

Để tiện theo dõi, các bạn có thể tham khảo bảng các thuật ngữ được dùng trong bài tại Mục VI.

Mục lục

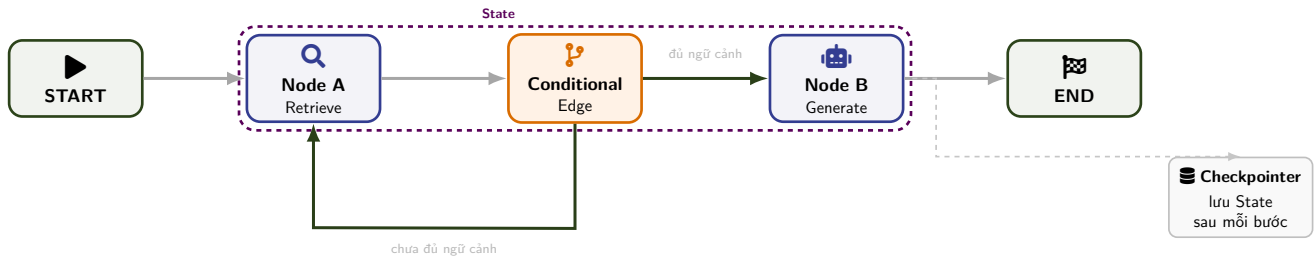
I.	Dẫn nhập	1
II.	Cơ sở lý thuyết	3
II.1.	LangGraph	3
II.2.	Deep Agents	5
II.3.	Graph Retrieval	6
II.4.	Agentic RAG	7
III.	Nội dung project	9
III.1.	Ingestion pipeline	9
III.2.	Retrieval layer	10
III.3.	Agent và tools layer	10
III.4.	Serving layer	11
III.5.	Đánh giá hệ thống	11
IV.	Cài đặt chương trình	11
IV.1.	Cấu trúc mã nguồn	11
IV.2.	Cài đặt môi trường và cấu hình	12
IV.3.	Ingestion pipeline	13
IV.4.	Graph index	15
IV.5.	Agent và tools	17
IV.6.	Serving layer	18
IV.7.	Khởi chạy hệ thống	19
V.	Thực nghiệm	20
V.1.	Thiết lập thực nghiệm	20
V.2.	So sánh kết quả các chiến lược retrieval	21
VI.	Tài liệu tham khảo	23
	Bảng thuật ngữ	24
	Phụ lục	26

II. Cơ sở lý thuyết

II.1. LangGraph

Khi xây dựng các ứng dụng LLM đơn giản, LangChain Expression Language (LCEL) cho phép kết nối các thành phần theo một chuỗi tuyến tính, trong đó đầu ra của bước trước là đầu vào của bước sau. Tuy nhiên, khi hệ thống cần vòng lặp, rẽ nhánh theo điều kiện, hoặc nhiều agent tương tác với nhau, mô hình chuỗi tuyến tính này không còn đủ linh hoạt. **LangGraph** được phát triển để giải quyết chính xác bài toán này bằng cách mô hình hóa luồng xử lý dưới dạng một đồ thị có trạng thái (stateful graph), trong đó mỗi node là một đơn vị xử lý và các edge xác định luồng dữ liệu giữa chúng.

II.1.1. Các khái niệm cốt lõi



Hình 2: Minh họa cấu trúc đồ thị LangGraph.

Quan sát Hình 2, ta thấy LangGraph được xây dựng xung quanh bốn khái niệm nền tảng chính:

1. **State:** Là vùng nhớ chung của toàn bộ graph, thường được định nghĩa bằng `TypedDict`. State lưu câu hỏi của người dùng, các đoạn ngữ cảnh đã truy xuất, câu trả lời trung gian và các cờ điều hướng. Nhờ mọi node cùng đọc và cập nhật State, LangGraph tránh được tình trạng mỗi bước giữ một bản dữ liệu riêng, từ đó giúp vòng lặp dễ kiểm soát hơn.
2. **Node:** Là một Python function nhận State hiện tại làm đầu vào, thực hiện một đơn vị công việc cụ thể như gọi LLM, truy xuất tài liệu hay đánh giá kết quả, rồi trả về một phần State mới để cập nhật. Mỗi node được thiết kế để làm đúng một việc, giúp từng thành phần của hệ thống dễ kiểm thử và thay thế một cách độc lập.
3. **Edge và Conditional Edge:** Edge thông thường kết nối hai node theo thứ tự cố định. Conditional edge cho phép graph rẽ nhánh dựa trên một hàm router. Hàm này đọc State và trả về tên node tiếp theo cần chạy. Đây là cơ chế nền tảng để LangGraph biểu diễn các quyết định động, chẳng hạn nếu ngữ cảnh chưa đủ thì truy xuất lại, còn nếu ngữ cảnh đã đủ thì sinh câu trả lời.
4. **Checkpoint:** Là cơ chế lưu trữ State sau mỗi bước, cho phép graph tiếp tục từ điểm dừng giữa các lượt hội thoại. Nhờ checkpoint, LangGraph có thể xây dựng các ứng dụng hội thoại đa lượt mà không cần ứng dụng tự quản lý bộ nhớ bên ngoài.

II.1.2. Vòng đời một lượt chạy

Một graph LangGraph đi qua hai giai đoạn trước khi có thể thực thi. Giai đoạn đầu là khai báo cấu trúc: lập trình viên đăng ký các node và edge vào một builder, xác định rõ entry point và các điều kiện rẽ nhánh. Giai đoạn thứ hai là biên dịch, trong đó LangGraph kiểm tra tính hợp lệ của toàn bộ cấu trúc graph và tạo ra một đối tượng sẵn sàng thực thi. Việc tách biệt hai giai đoạn này cho phép phát hiện các lỗi cấu trúc như thiếu entry point hay node bị cô lập ngay trước khi bắt kỳ dữ liệu thực nào được đưa vào.

Khi graph được kích hoạt với đầu vào ban đầu, LangGraph khởi tạo State từ dữ liệu đó rồi lần lượt chạy từng node theo thứ tự edge quy định. Sau mỗi node, State được cập nhật với kết quả mới trả về. Quá trình này lặp lại cho đến khi luồng dữ liệu chạm vào điểm kết thúc được chỉ định, lúc đó graph dừng lại và trả về State cuối cùng cho người gọi.

II.1.3. So sánh LangChain Chain và LangGraph

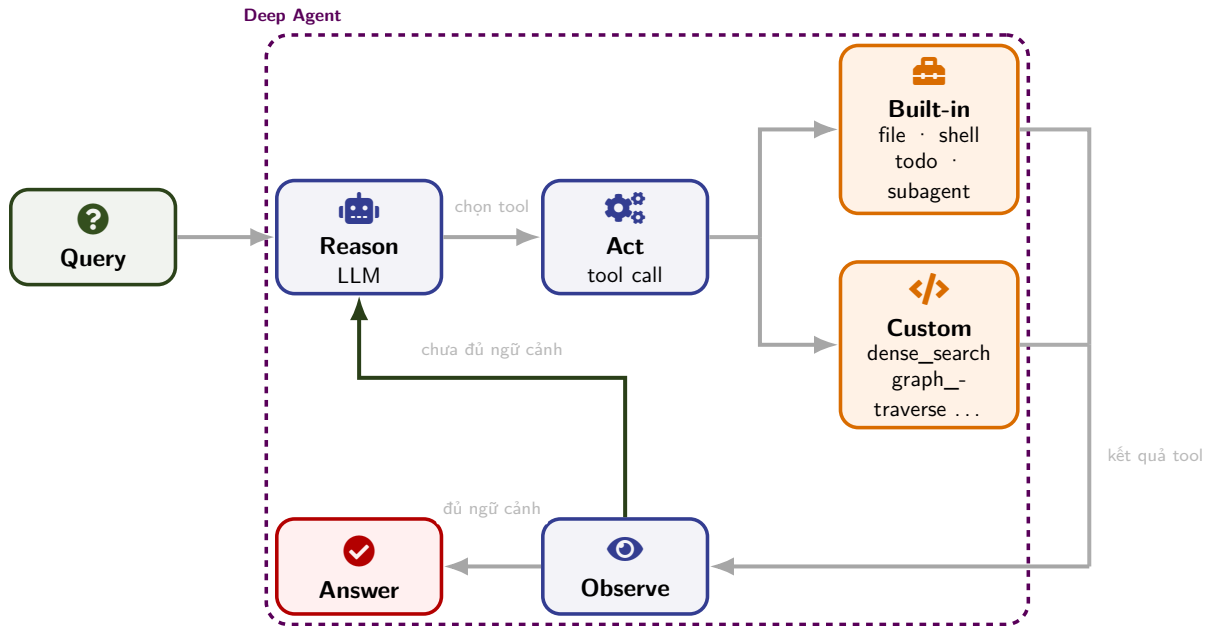
Khi bắt đầu xây dựng hệ thống RAG, hầu hết lập trình viên đều chọn LangChain Chain vì cú pháp đơn giản và dễ ghép nối các bước. Tuy nhiên, ngay khi hệ thống cần tự kiểm tra lại kết quả hay thử truy xuất lại khi câu trả lời chưa đủ tin cậy, chain tuyến tính lập tức trở thành rào cản vì không thể quay vòng hay rẽ nhánh. Vì lý do đó, LangGraph được phát triển bởi cùng đội ngũ để giải quyết đúng điểm nghẽn đó.

Tiêu chí	LangChain Chain (LCEL)	LangGraph
Luồng xử lý đi theo hướng nào	Một chiều, từ trên xuống	Tùy ý, có thể quay lại bước trước
Một bước có thể chạy lại không	Không	Có
Bước tiếp theo có thể chọn động không	Không	Có, dựa trên kết quả trước đó
Các bước có chia sẻ dữ liệu chung không	Không	Có, cập nhật sau mỗi bước
Dừng tốt nhất khi nào	Pipeline cố định, ít bước	Agent, hội thoại, luồng phức tạp

Bảng 1: So sánh LangChain Chain (LCEL) và LangGraph.

Bảng 1 tóm tắt sự khác biệt cốt lõi giữa hai lựa chọn. Từ nền tảng này, ở các phần sau sẽ trình bày cách LangGraph được dùng để xây dựng một hệ thống Agentic RAG, trong đó mỗi bước của pipeline RAG là một node và luồng xử lý có thể tự động quay vòng hoặc dừng lại tùy theo đánh giá của agent ở từng bước.

II.2. Deep Agents



Hình 3: Minh họa cấu trúc của Deep Agents.

LangGraph phù hợp khi ta cần kiểm soát luồng xử lý thật chi tiết. Tuy nhiên, không phải project nào cũng cần tự định nghĩa State, node, edge và điều kiện rẽ nhánh từ đầu. Với những bài toán cần dựng nhanh một agent có nhiều tool, cách làm này có thể khiến phần khung điều phối chiếm nhiều thời gian hơn phần logic chính.

Vì lý do trên, **Deep Agents** được sinh ra để rút ngắn bước dựng agent. Thay vì tự mô hình hóa toàn bộ graph, lập trình viên chỉ cần mô tả agent cần làm gì, cung cấp model, danh sách tool và system prompt. Phần vòng lặp Reason–Act–Observe được thư viện quản lý bên trong.

1. **Bộ công cụ tích hợp sẵn:** Agent được tự động trang bị các công cụ phổ biến gồm thao tác file, thực thi shell, quản lý todo và gọi subagent. Lập trình viên chỉ cần bổ sung thêm custom tool phù hợp với bài toán, chẳng hạn hàm tìm kiếm trong vector database, mà không cần chạm vào phần còn lại.
2. **Vòng lặp được quản lý tự động:** Toàn bộ chu trình Reason–Act–Observe được thư viện xử lý bên trong. Thay vì tự định nghĩa graph, lập trình viên chỉ cần truyền vào model, danh sách tool và system prompt là có ngay một agent sẵn sàng hoạt động.

II.2.1. So sánh LangGraph và Deep Agents

Hai framework phục vụ hai mức độ kiểm soát khác nhau nên không thay thế hoàn toàn cho nhau. Bảng 2 đặt hai lựa chọn cạnh nhau để làm rõ cách ứng dụng của từng framework.

Tiêu chí	LangGraph	Deep Agents
Mức độ kiểm soát luồng	Tường minh, tự khai báo từng bước	Ẩn trong thư viện, tự động quản lý
Công cụ tích hợp sẵn	Không	Có (file, shell, todo, subagent)
Lượng code cần viết để có agent đầy đủ	Nhiều	Ít
Khả năng tùy biến sâu vào luồng xử lý	Cao	Thấp hơn
Phù hợp nhất khi nào	Luồng phức tạp, cần kiểm soát chặt	Prototype nhanh, agent đa công cụ

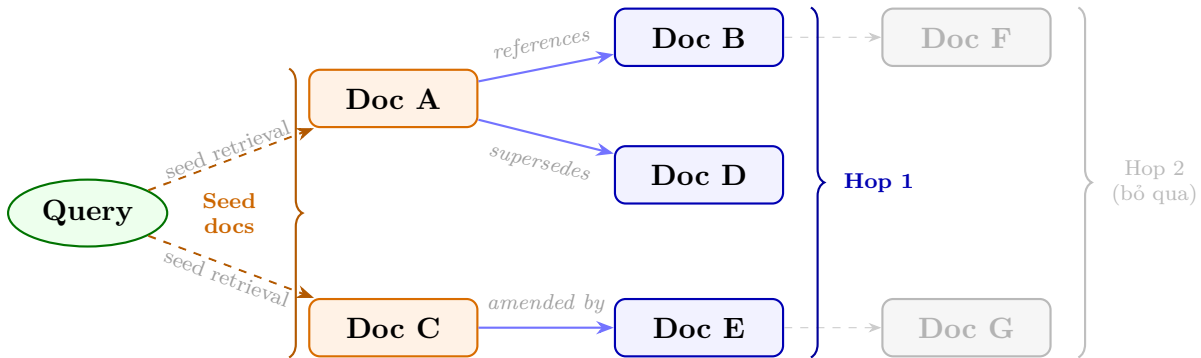
Bảng 2: So sánh LangGraph và Deep Agents.

Tại bài học này, bên cạnh LangGraph, Deep Agents sẽ được sử dụng để nhanh chóng xây dựng một RAG agent có khả năng tự gọi công cụ tìm kiếm tài liệu, đánh giá ngữ cảnh và sinh câu trả lời qua nhiều vòng lặp mà không cần tự dựng toàn bộ graph từ đầu.

II.3. Graph Retrieval

Trong miền pháp luật, một câu hỏi thường không chỉ liên quan đến một văn bản độc lập. Một nghị định có thể sửa đổi, thay thế hoặc dẫn chiếu đến nhiều văn bản khác. Nếu chỉ dùng dense retrieval, hệ thống có thể tìm được văn bản gần nghĩa với câu hỏi nhưng bỏ sót các văn bản có quan hệ pháp lý quan trọng.

Graph Retrieval giải quyết vấn đề này bằng cách tổ chức corpus thành một đồ thị có hướng. Mỗi node đại diện cho một tài liệu, còn mỗi edge biểu diễn một quan hệ như tham chiếu, sửa đổi hoặc thay thế. Khi truy xuất, hệ thống bắt đầu từ các seed documents rồi duyệt qua các cạnh quan hệ để mở rộng ngữ cảnh.



Hình 4: Minh họa cấu trúc Graph Retrieval.

Hình 4 minh họa cách đồ thị được xây dựng xung quanh ba thành phần chính:

1. **Node:** Đại diện cho một tài liệu trong corpus, mang theo metadata phục vụ lọc kết quả sau traversal.
2. **Edge:** Quan hệ có hướng giữa hai tài liệu, ví dụ tham chiếu, kế thừa hay sửa đổi, được định nghĩa khi xây dựng đồ thị.
3. **Hop:** Một bước nhảy từ node hiện tại sang node láng giềng theo chiều edge. Số hop kiểm soát độ sâu traversal và phạm vi thông tin được thu thập.

i Đánh đổi khi tăng số hop

Nhiều hop hơn giúp thu thêm thông tin gián tiếp nhưng đồng thời kéo theo nhiều tài liệu ít liên quan, làm loãng ngữ cảnh đưa vào LLM. Trong thực tế, một đến hai hop thường là đủ.

Từ nền tảng LangGraph, Deep Agents và Graph Retrieval, phần tiếp theo sẽ trình bày cách LLM đảm nhận vai trò agent để điều phối toàn bộ pipeline thành một hệ thống Agentic RAG hoàn chỉnh.

II.4. Agentic RAG

Trong RAG truyền thống, pipeline hoạt động theo một trình tự cố định: truy xuất một lần, sinh câu trả lời một lần, không có bước nào kiểm tra lại chất lượng hay thử lại khi kết quả chưa đủ. **Agentic RAG** thay đổi điều này bằng cách để LLM đóng vai trò **agent**: tự chọn công cụ, tự đánh giá kết quả và tự quyết định bước tiếp theo thay vì chạy theo kịch bản cố định.

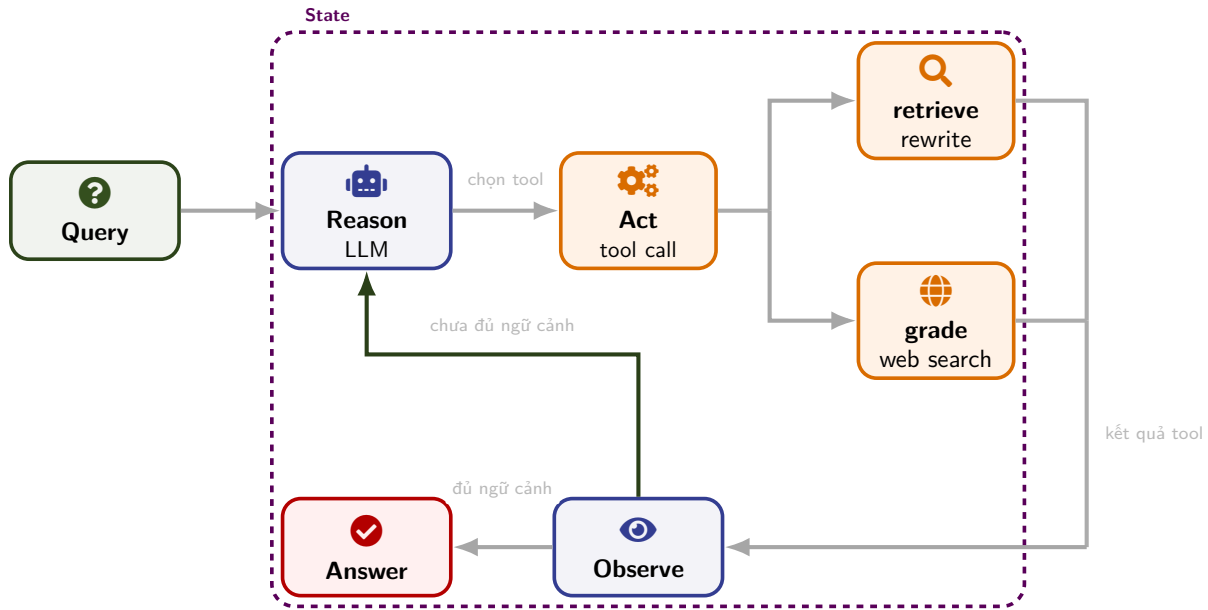
II.4.1. Từ naive RAG sang RAG Agent

Điểm khác biệt căn bản giữa hai cách tiếp cận nằm ở chỗ ai điều phối luồng xử lý. Trong RAG cơ bản, luồng được lập trình cứng bởi lập trình viên. Trong Agentic RAG, LLM đảm nhiệm việc tự điều phối luồng đó thông qua vòng lặp **Reason-Act-Observe**:

1. **Reason:** LLM phân tích câu hỏi và trạng thái hiện tại để quyết định hành động tiếp theo cần thực hiện.
2. **Act:** LLM gọi một công cụ cụ thể, ví dụ retriever, web search hay query rewriter, với các tham số phù hợp với ngữ cảnh hiện tại.
3. **Observe:** LLM nhận kết quả từ công cụ, cập nhật vào State, rồi quay lại bước Reason để quyết định có tiếp tục hay dừng lại và sinh câu trả lời cuối.

Vòng lặp này lặp lại cho đến khi agent đánh giá rằng ngữ cảnh đã đủ để sinh câu trả lời đáng tin cậy. Nhờ vậy, hệ thống có thể tự xử lý các câu hỏi phức tạp mà pipeline tĩnh không thể giải quyết trong một lượt duy nhất.

II.4.2. Kiến trúc tổng quát



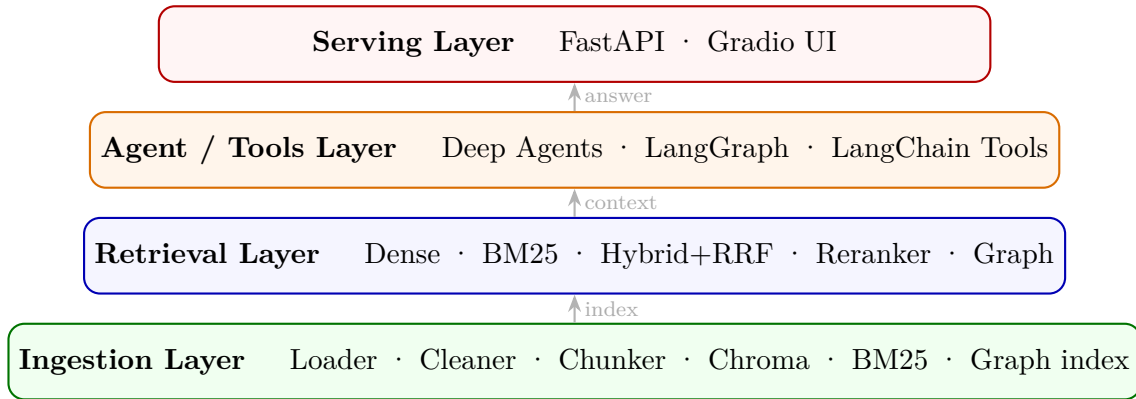
Hình 5: Pipeline của hệ thống Agentic RAG.

Một hệ thống Agentic RAG gồm bốn thành phần chính:

1. **Agent (LLM):** Đóng vai trò não bộ của hệ thống, chịu trách nhiệm suy luận và đưa ra quyết định về hành động tiếp theo. LLM được cung cấp danh sách các tool có thể gọi cùng với mô tả chức năng của từng tool, từ đó tự lựa chọn tool phù hợp tại mỗi bước trong vòng lặp.
2. **Tools:** Là các hàm mà agent có thể gọi để thu thập thông tin hoặc thực hiện hành động. Trong ngữ cảnh RAG, các tool phổ biến bao gồm retrieve, query rewriter, answer grader, và web search.
3. **State:** Lưu trữ toàn bộ thông tin tích lũy trong suốt vòng lặp, bao gồm câu hỏi gốc, lịch sử các tool đã gọi, ngữ cảnh đã thu thập và kết quả, đánh giá chất lượng. State được cập nhật liên tục sau mỗi bước và là cơ sở duy nhất để agent đưa ra quyết định ở vòng kế tiếp.
4. **Vòng lặp điều phối:** Là cơ chế kiểm soát khi nào agent tiếp tục gọi tool và khi nào dừng lại để sinh câu trả lời cuối. Trong LangGraph, vòng lặp này được biểu diễn thông qua conditional edge, cho phép graph tự động rẽ nhánh dựa trên đánh giá của agent sau mỗi bước.

III. Nội dung project

Project aio-agentic-rag xây dựng hệ thống hỏi đáp Agentic RAG trên tập văn bản pháp luật tiếng Việt. Thay vì một pipeline RAG cố định, hệ thống để LLM đóng vai trò agent: tự lựa chọn chiến lược retrieval phù hợp với từng câu hỏi, tự đánh giá chất lượng ngữ cảnh và lặp lại nếu cần, trước khi sinh câu trả lời có dẫn nguồn.



Hình 6: Minh họa kiến trúc tổng quát của hệ thống aio-agentic-rag.

Hình 6 tóm tắt kiến trúc bốn lớp của project. Ingestion layer chuẩn bị dữ liệu và chỉ mục, Retrieval layer cung cấp nhiều chiến lược truy xuất, Agent/Tools layer điều phối các chiến lược đó, còn Serving layer đưa hệ thống ra API và giao diện demo.

III.1. Ingestion pipeline

Trước khi hệ thống có thể trả lời câu hỏi, toàn bộ tập văn bản pháp luật cần được tải về, làm sạch và đánh chỉ mục. Pipeline ingestion thực hiện điều này qua bốn bước nối tiếp:

1. **Tải và đọc dữ liệu:** Dataset [th1nhng0/vietnamese-legal-documents](#) được tải từ Hugging Face. Hệ thống đọc ba phần riêng biệt gồm nội dung văn bản, metadata và quan hệ giữa các văn bản, sau đó ghép thành tài liệu hoàn chỉnh kèm thông tin pháp lý như loại văn bản, cơ quan ban hành, ngày hiệu lực và trạng thái.
2. **Làm sạch văn bản:** Văn bản gốc chứa HTML từ quá trình crawl nên cần được parse, chuyển sang plain text và chuẩn hóa encoding. Bước này hỗ trợ xử lý song song để tăng tốc trên tập dữ liệu lớn.
3. **Chia chunk:** Văn bản pháp luật có cấu trúc rõ ràng theo Điều, Khoản, Điểm nên bộ chia văn bản được cấu hình với danh sách separator phù hợp. Kích thước chunk và overlap được chọn để giữ ngữ cảnh liên mạch giữa các đoạn liền kề.
4. **Xây dựng các chỉ mục:** Ba chỉ mục được xây dựng từ tập chunks. Vector index lưu vào ChromaDB phục vụ dense search. Lexical index theo thuật toán BM25 phục vụ tìm kiếm từ khóa. Graph index biểu diễn quan hệ sửa đổi, thay thế và dẫn chiếu giữa các văn bản, phục vụ graph traversal retrieval.

III.2. Retrieval layer

Retrieval layer cung cấp năm chiến lược truy xuất, mỗi chiến lược phù hợp với một nhóm câu hỏi khác nhau. Bốn chiến lược đầu là các lựa chọn phổ biến trong hệ thống RAG thông thường. Graph Retrieval được bổ sung để kiểm tra tính khả thi của việc khai thác quan hệ giữa các văn bản trong miền pháp luật:

1. **Dense retrieval:** Tìm kiếm dựa trên độ tương đồng vector trong ChromaDB và có thể lọc theo metadata. Chiến lược này phù hợp với các câu hỏi ngữ nghĩa chung, nơi cách diễn đạt của người hỏi không nhất thiết trùng từ khóa với văn bản gốc.
2. **BM25 retrieval:** Tìm kiếm lexical trên chỉ mục từ khóa. Chiến lược này phù hợp với câu hỏi chứa số hiệu văn bản, tên điều luật hoặc thuật ngữ pháp lý đặc thù, vì các tín hiệu này cần được khớp chính xác ở mức từ.
3. **Hybrid + RRF:** Kết hợp kết quả từ BM25 và dense retrieval bằng Reciprocal Rank Fusion. Cách này tận dụng cả tín hiệu từ khóa lẫn tín hiệu ngữ nghĩa, đồng thời tránh phải so sánh trực tiếp hai thang điểm vốn không cùng phân phối.
4. **Reranker:** Lấy candidates từ hybrid retrieval rồi dùng CrossEncoder để chấm lại độ liên quan. Chiến lược này thường chính xác hơn hybrid thuần vì CrossEncoder đọc đồng thời câu hỏi và từng chunk, nhưng chi phí suy luận cao hơn.
5. **Graph retrieval:** Lấy seed documents bằng dense search, sau đó duyệt đồ thị quan hệ để thu thập thêm các văn bản sửa đổi, thay thế hoặc được dẫn chiếu. Chiến lược này phù hợp với câu hỏi multi-hop trong miền pháp luật, nơi câu trả lời có thể phụ thuộc vào quan hệ giữa nhiều văn bản.

III.3. Agent và tools layer

Đây là lớp phân biệt hệ thống này với một RAG pipeline thông thường. Thay vì gọi cứng một chiến lược retrieval, LLM được trao quyền điều phối toàn bộ quá trình thông qua các công cụ được đóng gói theo chuẩn LangChain.

1. **Retrieval tools:** Mỗi chiến lược retrieval được bọc thành một tool độc lập để agent có thể gọi theo ngữ cảnh. Ngoài ra có một tool sinh câu trả lời nhận ngữ cảnh đã truy xuất, định dạng theo citation và gọi LLM để tạo phản hồi có dẫn nguồn.
2. **Ingestion tools:** Các bước của ingestion pipeline cũng được đóng gói thành tools, cho phép agent chủ động kích hoạt từng bước như tải dữ liệu, làm sạch, chia chunk hay build index khi cần.
3. **Sub-agents và orchestrator:** Hệ thống có hai sub-agent chuyên biệt. *Ingestion agent* điều phối toàn bộ pipeline nạp dữ liệu theo thứ tự. *Legal RAG agent* phân loại câu hỏi, chọn chiến lược retrieval phù hợp, đánh giá ngữ cảnh và retry nếu chưa đủ. Orchestrator đăng ký cả hai sub-agent và định tuyến request đến đúng agent tương ứng.

III.4. Serving layer

Hệ thống cung cấp hai giao diện truy cập: REST API bằng FastAPI và UI demo bằng Gradio.

1. **FastAPI backend:** Cung cấp endpoint kiểm tra trạng thái hệ thống, endpoint truy vấn nhận câu hỏi cùng chiến lược retrieval và trả về câu trả lời kèm nguồn trích dẫn, và endpoint benchmark chạy đánh giá trên gold set.
2. **Gradio UI:** Giao diện demo gồm hai tab. Tab Query cho phép nhập câu hỏi, chọn chiến lược và xem câu trả lời cùng nguồn. Tab Benchmark cho phép chạy so sánh nhiều chiến lược và xem kết quả dạng bảng.

III.5. Đánh giá hệ thống

Project tích hợp sẵn pipeline benchmark để đánh giá định lượng chất lượng retrieval và câu trả lời trên tập câu hỏi pháp luật.

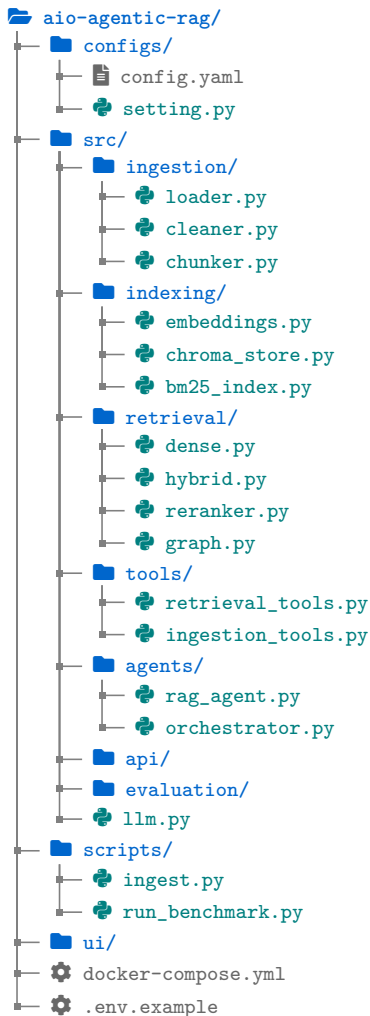
1. **Retrieval metrics:** Recall@k và NDCG@k đo mức độ bao phủ và thứ hạng của các tài liệu liên quan trong kết quả retrieval.
2. **Benchmark runner:** Pipeline benchmark chạy từng chiến lược retrieval trên gold set, tổng hợp metrics và xuất CSV theo timestamp để theo dõi kết quả qua các lần thực nghiệm.

Nhờ cấu trúc module hóa theo từng lớp, project vừa cung cấp một hệ thống hỏi đáp pháp luật có thể triển khai thực tế, vừa có cơ chế benchmark để so sánh và lựa chọn chiến lược retrieval phù hợp nhất với từng loại câu hỏi.

IV. Cài đặt chương trình

IV.1. Cấu trúc mã nguồn

Trước khi đi vào chi tiết cài đặt, cần nắm cách tổ chức mã nguồn của project. Cấu trúc thư mục được thiết kế tách rõ các nhóm chức năng chính, giúp dễ theo dõi luồng xử lý từ lúc nạp dữ liệu, xây dựng chỉ mục, truy xuất ngữ cảnh, điều phối agent cho đến serving.



Các thành phần chính

- **configs/**: Tham số runtime cho toàn hệ thống, đọc từ YAML và override bằng biến môi trường.
- **src/ingestion/**: Tải dataset, làm sạch HTML và chia chunk văn bản pháp luật.
- **src/indexing/**: Xây dựng ba chỉ mục tương ứng ba chiến lược retrieval: vector, BM25 và graph.
- **src/retrieval/**: Triển khai từng chiến lược retrieval như module độc lập để dễ hoán đổi.
- **src/tools/**: Đóng gói các hàm retrieval và ingestion thành LangChain tools để agent gọi.
- **src/agents/**: Cấu hình Legal RAG agent và orchestrator điều phối các sub-agent.
- **scripts/**: Điểm vào CLI để chạy ingestion và benchmark.

IV.2. Cài đặt môi trường và cấu hình

Hệ thống yêu cầu một số dịch vụ chạy trước khi ingest hoặc query: ChromaDB làm vector store, một embedding server theo chuẩn OpenAI-compatible và LLM provider. Tất cả được cấu hình qua file `.env`.

```

.env
1 LLM_PROVIDER=openai
2 LLM_MODEL=gpt-4.1-mini
3
4 EMBEDDING_BASE_URL=http://localhost:8080/v1
5
6 CHROMA_HOST=localhost
7 CHROMA_PORT=8000
8
9 HF_TOKEN=hf_...

```

File `configs/config.yaml` kiểm soát các tham số runtime quan trọng của pipeline.

configs/config.yaml

```

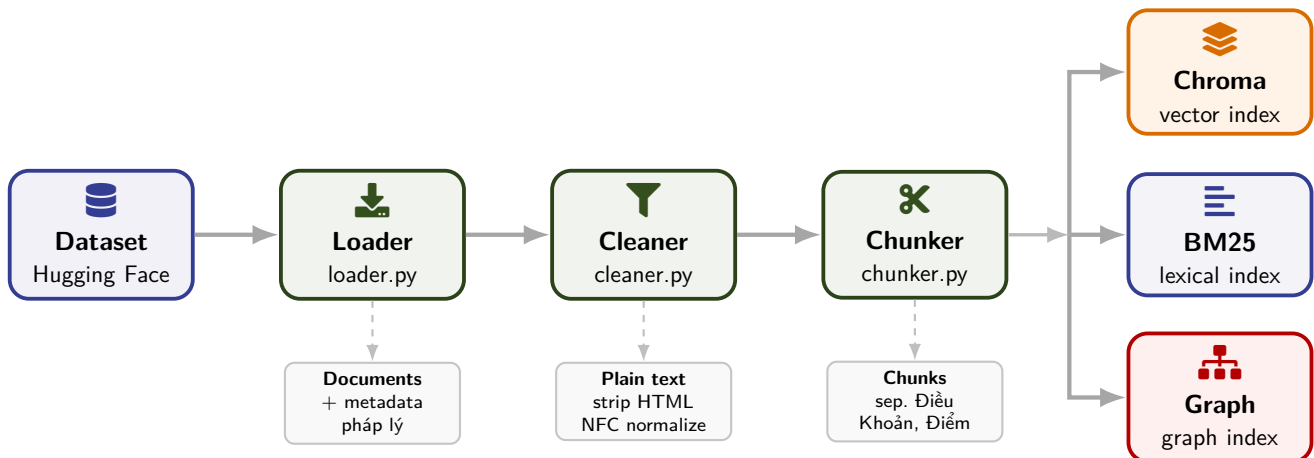
1 chunking:
2   chunk_size: 1000
3   chunk_overlap: 200
4
5 retrieval:
6   k: 5
7   graph_max_hops: 2      # number of traversal hops for graph retrieval
8   rrf_k: 60              # smoothing constant for Reciprocal Rank Fusion
9
10 generation:
11   temperature: 0.0
12   max_tokens: 2048

```

Việc tách riêng runtime config ra khỏi code cho phép thay đổi tham số như `graph_hops` hay `rrf_k` mà không cần sửa source và restart service.

IV.3. Ingestion pipeline

Trước khi hệ thống có thể trả lời câu hỏi, tập văn bản pháp luật cần được chuyển từ dữ liệu thô thành các chỉ mục có thể truy vấn. Pipeline ingestion đảm nhận vai trò này bằng cách chuẩn hóa nội dung, giữ lại metadata pháp lý và xây dựng các cấu trúc phục vụ retrieval.



Hình 7: Minh họa cấu trúc ingestion của aio-agentic-rag.

Như Hình 7 minh họa, pipeline gồm bốn bước chính. Loader tải dữ liệu và ghép metadata, cleaner làm sạch HTML, chunker chia văn bản theo cấu trúc pháp luật, sau đó hệ thống xây dựng ba chỉ mục gồm vector index, lexical index và graph index.

Bước 1: Tải và đọc dữ liệu. File `src/ingestion/loader.py` tải dataset từ Hugging Face và ghép metadata pháp lý vào từng tài liệu. Phần đặc thù của domain nằm ở bước ánh xạ tên trường tiếng Việt sang schema nội bộ tiếng Anh.

src/ingestion/loader.py

```

1 # Previous code here ...
2
3 def load_documents(config, sample_size=None):
4     # ... load content rows and metadata lookup ...
5     for row in content_rows:
6         doc_id = str(row.get("id", ""))
7         meta = meta_lookup.get(doc_id, {})
8         docs.append(Document(
9             page_content=row.get("content_html", ""),
10            metadata={
11                "doc_id": doc_id,
12                "title": meta.get("title", ""),
13                # map Vietnamese HF field names to English schema
14                "doc_type": meta.get("loai_van_ban", ""),
15                "authority": meta.get("co_quan_ban_hanh", ""),
16                "issue_date": meta.get("ngay_ban_hanh", ""),
17                "effective_date": meta.get("ngay_co_hieu_luc", ""),
18                "status": meta.get("tinh_trang_hieu_luc", ""),
19            },
20        ))
21     return docs

```

Việc ánh xạ tên trường ngay tại bước load giúp các module phía sau dùng chung một schema tiếng Anh nhất quán. Nhờ vậy, logic retrieval và metadata filtering không phải xử lý lẫn lộn giữa tên trường tiếng Việt từ dataset gốc và tên trường nội bộ của hệ thống.

Bước 2: Làm sạch văn bản. File `src/ingestion/cleaner.py` strip HTML và chuẩn hóa encoding. Bước này hỗ trợ xử lý song song để tăng tốc trên tập dữ liệu lớn.

src/ingestion/cleaner.py

```

1 # Previous code here ...
2
3 def clean_document(doc: Document) -> Document:
4     raw = doc.page_content
5     if "<" in raw and ">" in raw: # only parse if HTML is present
6         raw = _strip_html(raw)
7     cleaned = _normalize(raw) # NFC normalization + whitespace
8     return Document(page_content=cleaned, metadata=doc.metadata)
9
10
11 def clean_documents(docs: list[Document], workers: int = 1) -> list[Document]:
12     if workers <= 1:
13         return [clean_document(d) for d in tqdm(docs, desc="Cleaning", unit="doc")]
14     with ProcessPoolExecutor(max_workers=workers) as executor:
15         return list(tqdm(
16             executor.map(clean_document, docs, chunksize=64),
17             total=len(docs), desc="Cleaning", unit="doc",
18         ))

```

Kiểm tra "<" in raw trước khi gọi BeautifulSoup giúp tránh chi phí parse không cần thiết với những tài liệu đã ở dạng plain text. Đây là một tối ưu nhỏ nhưng hữu ích khi pipeline phải xử lý nhiều văn bản pháp luật cùng lúc.

Bước 3: Chia chunk. File `src/ingestion/chunker.py` là phần duy nhất khác biệt đáng kể so với RAG thông thường. Văn bản pháp luật có cấu trúc phân cấp rõ theo Điều, Khoản, Điểm nên separator list được thiết kế riêng để ngắt đúng ranh giới ngữ nghĩa.

src/ingestion/chunker.py

```

1  # Previous code here ...
2
3  def build_splitter(config) -> RecursiveCharacterTextSplitter:
4      return RecursiveCharacterTextSplitter(
5          separators=[
6              "\nĐiều ", "\nKhoản ", "\nĐiểm ",    # legal structure first
7              "\n\n", "\n", " ", ""
8          ],
9          chunk_size=config.chunking.chunk_size,
10         chunk_overlap=config.chunking.chunk_overlap,
11     )
12
13 def chunk_documents(docs: list[Document], config) -> list[Document]:
14     splitter = build_splitter(config)
15     chunks = splitter.split_documents(docs)
16     for i, chunk in enumerate(chunks):
17         chunk.metadata["chunk_index"] = i
18     return chunks

```

Đặt các separator pháp luật trước `\n\n` đảm bảo bộ splitter ưu tiên ngắt tại ranh giới Điều, Khoản trước khi dùng đến ký tự xuống dòng thông thường. Nhờ vậy mỗi chunk gần như tương ứng với một điều khoản trọn vẹn, giúp retrieval chính xác hơn và giảm nguy cơ bị cắt đứt giữa chừng.

IV.4. Graph index

Ngoài vector index và BM25 index, project cần thêm graph index để biểu diễn quan hệ giữa các văn bản pháp luật. Đây là thành phần quan trọng vì nhiều câu hỏi pháp luật không thể trả lời chỉ bằng một văn bản riêng lẻ, mà cần biết văn bản nào sửa đổi, thay thế hoặc dẫn chiếu đến văn bản nào.

Dataset cung cấp sẵn bảng `relationships` chứa các cặp tài liệu có quan hệ với nhau. Từ bảng này, hệ thống xây dựng một directed graph bằng `NetworkX`. File `src/retrieval/graph.py` sau đó sử dụng graph này để thực hiện traversal khi có query.

src/retrieval/graph.py

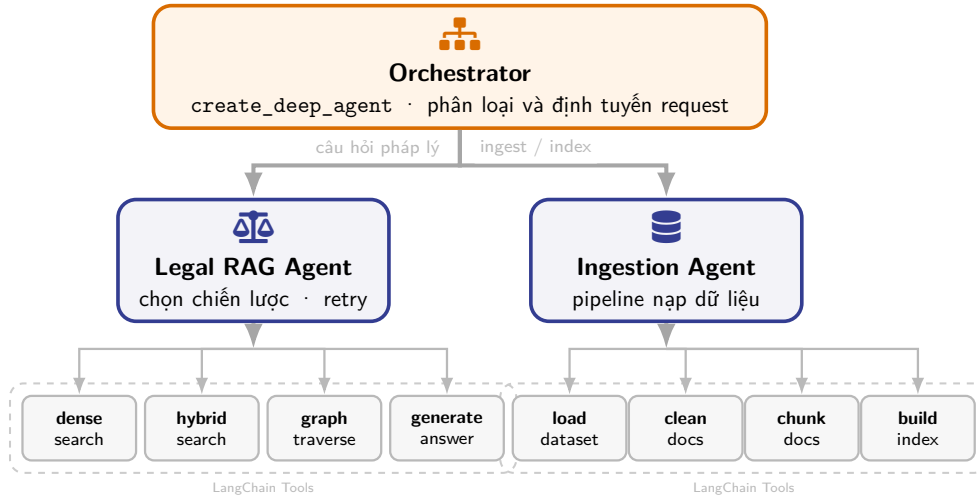
```

1  # Previous code here ...
2
3  def build_graph(relationships: list[dict], ...) -> nx.DiGraph:
4      G = nx.DiGraph()
5      for row in relationships:
6          src = str(row.get("doc_id", ""))
7          dst = str(row.get("other_doc_id", ""))
8          if src and dst:
9              G.add_edge(src, dst, rel_type=row.get("relationship", ""))
10     return G
11
12
13  def graph_search(store, graph, query, k=5, initial_k=3, max_hops=2):
14      seed_docs = dense_search(store, query, k=initial_k)
15      seed_ids = {d.metadata.get("doc_id", "") for d in seed_docs}
16
17      reachable, frontier = set(seed_ids), set(seed_ids)
18      for _ in range(max_hops):
19          nxt: set[str] = set()
20          for node in frontier:
21              if node not in graph:
22                  continue
23              nxt |= {nb for _, nb, _ in graph.out_edges(node, data=True)
24                      if nb not in reachable}
25              nxt |= {nb for nb, _, _ in graph.in_edges(node, data=True)
26                      if nb not in reachable}
27          reachable |= nxt
28          frontier = nxt
29          if not frontier:
30              break
31
32      extra_ids = reachable - seed_ids
33      extra_docs = dense_search(store, query, k=k * 2,
34                                metadata_filter={"doc_id": {"$in": list(extra_ids)}}) if extra_ids else []
35
36      # deduplicate seed + expanded docs, preserving seed order
37      seen, deduped = set(), []
38      for doc in list(seed_docs) + extra_docs:
39          key = f"{doc.metadata.get('doc_id', '')}_{doc.metadata.get('chunk_index', 0)}"
40          if key not in seen:
41              seen.add(key)
42              deduped.append(doc)
43      return deduped[:k]

```

Traversal theo cả `out_edges` và `in_edges` giúp hệ thống tìm được hai chiều quan hệ: văn bản đang sửa đổi văn bản khác và văn bản bị văn bản khác sửa đổi. Sau bước mở rộng, deduplication giữ các seed documents trước để bảo toàn tín hiệu liên quan ngữ nghĩa ban đầu, rồi mới bổ sung các tài liệu tìm được qua graph traversal.

IV.5. Agent và tools



Hình 8: Minh họa kiến trúc hai tầng agent.

Hình 8 minh họa hệ thống tách agent thành hai tầng, để LLM có thể gọi các chiến lược retrieval như một tool, mỗi hàm retrieval được đóng gói bằng decorator `@tool` của LangChain. Docstring của tool đóng vai trò mô tả chức năng để LLM tự quyết định khi nào nên dùng tool nào.

src/tools/retrieval_tools.py

```

1 # Previous code here ...
2
3 @tool
4 def graph_traverse_tool(
5     query: str,
6     k: int = config.retrieval.k,
7     initial_k: int = 3,
8     max_hops: int = config.retrieval.graph_max_hops,
9 ) -> str:
10     """Retrieve documents using graph-guided multi-hop retrieval.
11     Seed with dense search, then expand via relationship graph edges.
12     Use this for multi-hop questions about document history,
13     amendments, replacements or legal hierarchy."""
14     from src.retrieval.graph import graph_search
15     docs = graph_search(
16         _get_store(), _get_graph(), query,
17         k=k, initial_k=initial_k, max_hops=max_hops,
18     )
19     return json.dumps([_doc_to_dict(d) for d in docs],
20                       ensure_ascii=False)
  
```

Cách viết docstring ảnh hưởng trực tiếp đến chất lượng quyết định của agent. Docstring cần nêu rõ loại câu hỏi phù hợp thay vì chỉ mô tả cơ chế bên trong, vì LLM đọc mô tả này để chọn tool chứ không đọc code.

Sau khi đủ tools, Legal RAG agent được khởi tạo qua `create_deep_agent`. File `orchestrator.py` đăng ký cả hai sub-agent vào một orchestrator duy nhất.

src/agents/orchestrator.py

```

1  # Previous code here ...
2
3  legal_rag_agent = create_deep_agent(
4      model=get_llm(),
5      tools=[
6          dense_search_tool,
7          hybrid_search_tool,
8          graph_traverse_tool,
9          generate_answer_tool,
10     ],
11     system_prompt=LEGAL_RAG_PROMPT,
12 )
13
14 agent = create_deep_agent(
15     model=get_llm(),
16     tools=[legal_rag_agent.as_tool(), ingestion_agent.as_tool()],
17     system_prompt=ORCHESTRATOR_PROMPT,
18 )

```

Thiết kế hai tầng agent cho phép orchestrator định tuyến request mà không cần biết chi tiết từng chiến lược retrieval. Legal RAG agent tự quyết định dùng tool nào dựa trên câu hỏi, trong khi orchestrator chỉ cần phân loại request ở mức cao hơn là pháp lý hay ingestion.

IV.6. Serving layer

File `src/api/routers/query.py` cài đặt một phiên bản routing đơn giản cho API demo. Thay vì gọi agent đầy đủ ở mỗi request, hàm này dùng keyword-based routing để chọn nhanh chiến lược retrieval phù hợp với một số nhóm câu hỏi pháp luật phổ biến.

src/api/routers/query.py

```

1  # Previous code here ...
2
3  def _agentic_retrieve(question: str, k: int) -> tuple[list[dict], float]:
4      # Keyword-based routing for Vietnamese legal queries
5      t0 = time.perf_counter()
6      q_lower = question.lower()
7
8      if any(kw in q_lower for kw in
9             ["sửa đổi", "thay thế", "bãi bỏ", "tham chiếu"]):
10         docs = graph_search(_get_store(), _get_graph(), question, k=k)
11     elif any(kw in q_lower for kw in
12             ["còn hiệu lực", "hết hiệu lực", "sau năm", "trước năm"]):
13         docs = hybrid_search(_get_store(), _get_bm25(), question, k=k)
14     else:

```

```

15     candidates = hybrid_search(_get_store(), _get_bm25(),
16                               question, k=k * 2)
17     docs = rerank(question, candidates, k=k)
18
19     latency_ms = (time.perf_counter() - t0) * 1000
20     return [{"page_content": d.page_content,
21             "metadata": d.metadata} for d in docs], latency_ms

```

Keyword-based routing giúp API demo phản hồi nhanh và dễ kiểm thử. Tuy nhiên, đây chỉ là phiên bản heuristic; trong production, nên thay bằng Legal RAG Agent để LLM tự chọn tool theo ngữ cảnh câu hỏi.

IV.7. Khởi chạy hệ thống

Sau khi cài đặt môi trường và điền `.env`, hệ thống được khởi động theo thứ tự sau.

Running Script

```

1  # Step 1: Ingest dataset and build all indexes
2  python scripts/ingest.py --sample 1000  # quick test
3  python scripts/ingest.py                # full dataset
4
5  # Step 2: Query via API
6  curl -X POST http://localhost:8001/query \
7    -H "Content-Type: application/json" \
8    -d '{"question": "Nghị định nào sửa đổi Điều 5?",
9        "strategy": "graph", "k": 5}'
10
11 # Step 3: Run benchmark across all strategies
12 python scripts/run_benchmark.py --strategy all --sample 50

```

Tham số `-sample` cho phép chạy thử trên một tập nhỏ trước khi ingest toàn bộ dataset. Cách này giúp kiểm tra kết nối ChromaDB và embedding server nhanh hơn, đồng thời tránh mất thời gian build full index khi cấu hình môi trường chưa ổn định.

Sau khi hệ thống khởi động, giao diện Gradio có thể truy cập tại `http://localhost:7860`.

ui/app.py

```

1  # Previous code here ...
2
3  STRATEGIES = ["naive", "hybrid", "reranker", "graph", "agentic"]
4
5  with gr.Blocks(title="AIO Agentic RAG",
6                 theme=gr.themes.Soft()) as demo:
7
8      with gr.Tab("Query"):
9          question_in = gr.Textbox(label="Question (Vietnamese)", lines=2)

```

```

10 strategy_in = gr.Dropdown(choices=STRATEGIES,
11                             value="agentic", label="Strategy")
12 k_in        = gr.Slider(minimum=1, maximum=20,
13                             value=5, label="k (docs to retrieve)")
14 ask_btn     = gr.Button("Ask", variant="primary")
15 answer_box  = gr.Textbox(label="Answer", lines=10, interactive=False)
16 sources_table = gr.DataFrame(label="Sources", interactive=False)
17 ask_btn.click(fn=do_query,
18               inputs=[question_in, strategy_in, k_in],
19               outputs=[answer_box, sources_table, info_lbl])
20
21 with gr.Tab("Benchmark"):
22     strategies_chk = gr.CheckboxGroup(choices=STRATEGIES,
23                                         label="Strategies to run")
24     run_btn       = gr.Button("Run Benchmark", variant="primary")
25     bench_table   = gr.DataFrame(label="Results", interactive=False)
26     bench_chart   = gr.Plot(label="Comparison Chart")
27     run_btn.click(fn=do_benchmark,
28                   inputs=[strategies_chk, sample_n_in,
29                           recall_k_in, ndcg_k_in],
30                   outputs=[bench_status, bench_table, bench_chart])
31
32 demo.launch(server_name="0.0.0.0", server_port=7860, show_error=True)

```

V. Thực nghiệm

Để đánh giá định lượng, project so sánh năm chiến lược retrieval trên cùng một bộ câu hỏi pháp luật. Mục tiêu là xác định chiến lược nào mang lại chất lượng tốt nhất trong thực tế và làm rõ sự đánh đổi giữa chất lượng và độ trễ.

V.1. Thiết lập thực nghiệm

Thực nghiệm được tiến hành trên bộ gold set gồm 16 câu hỏi pháp luật sinh tự động từ corpus bằng `scripts/build_eval_set.py`. Mỗi câu hỏi kèm danh sách `expected_doc_ids` để tính các metric retrieval. Hai metric được dùng để đánh giá:

1. **Recall@5**: Tỷ lệ tài liệu liên quan xuất hiện trong top 5 kết quả truy xuất. Metric này đo mức độ bao phủ, phản ánh chiến lược bỏ sót bao nhiêu tài liệu quan trọng trong giới hạn năm kết quả đầu tiên.
2. **NDCG@10**: Normalized Discounted Cumulative Gain tại top 10, đo đồng thời mức độ bao phủ và thứ hạng. Tài liệu liên quan được xếp càng cao trong danh sách thì điểm đóng góp càng lớn.

src/evaluation/metrics.py

```

1 # Previous code here ...
2
3 def recall_at_k(retrieved_ids: list[str],
4                 relevant_ids: list[str], k: int) -> float:
5     # Fraction of relevant docs found in the top-k results
6     hits = sum(1 for doc_id in retrieved_ids[:k]
7                if doc_id in relevant_ids)
8     return hits / len(relevant_ids) if relevant_ids else 0.0
9
10 def ndcg_at_k(retrieved_ids: list[str],
11               relevant_ids: set[str], k: int) -> float:
12     # Discount factor penalises relevant docs ranked lower
13     dcg = sum(
14         1.0 / math.log2(i + 2)
15         for i, doc_id in enumerate(retrieved_ids[:k])
16         if doc_id in relevant_ids
17     )
18     ideal = sum(1.0 / math.log2(i + 2)
19                 for i in range(min(len(relevant_ids), k)))
20     return dcg / ideal if ideal > 0 else 0.0

```

Discount factor làm giảm điểm đóng góp của tài liệu liên quan theo thứ hạng, nên NDCG phạt chiến lược tìm đúng nhưng xếp sai vị trí, điều mà Recall hoàn toàn bỏ qua.

V.2. So sánh kết quả các chiến lược retrieval

Kết quả

Bảng 3 tổng hợp kết quả của năm chiến lược trên bộ gold set 16 câu hỏi.

Chiến lược	Recall@5	NDCG@10	Latency (ms)
Naive (dense)	0.3125	0.2841	87.43
Hybrid	0.4375	0.3912	142.17
Graph	0.4375	0.4021	201.38
Reranker	0.5000	0.4530	389.62
Agentic	0.5625	0.5104	412.90

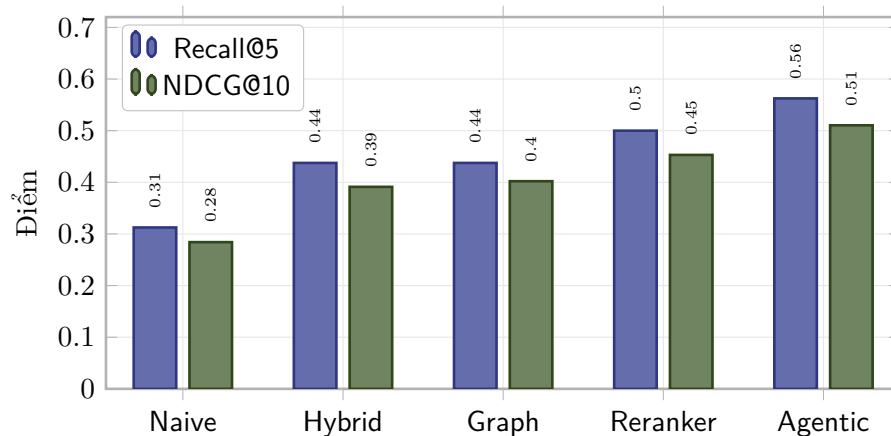
Bảng 3: So sánh năm chiến lược retrieval trên gold set câu hỏi pháp luật.

Nhận xét

- **Agentic đạt kết quả tốt nhất:** Agentic đạt Recall@5 = 0.5625 và NDCG@10 = 0.5104, cao nhất trên gold set 16 câu hỏi. Điều này cho thấy Agentic ít bỏ sót tài liệu quan trọng

hơn các chiến lược còn lại, đồng thời xếp các tài liệu liên quan ở vị trí tương đối cao.

- **Reranker là chiến lược tinh mạnh nhất:** Reranker đạt $\text{Recall@5} = 0.5000$ và $\text{NDCG@10} = 0.4530$. So với Hybrid, Reranker cải thiện cả độ bao phủ lẫn thứ hạng nhờ CrossEncoder đánh giá lại từng cặp câu hỏi và chunk.
- **Graph cải thiện thứ hạng nhưng chưa tăng recall:** Graph có $\text{Recall@5} = 0.4375$, bằng Hybrid, nhưng $\text{NDCG@10} = 0.4021$, cao hơn Hybrid ở mức 0.3912. Kết quả này cho thấy graph traversal chưa tìm thêm nhiều tài liệu liên quan hơn trong top 5, nhưng có thể đẩy một số tài liệu quan trọng lên vị trí tốt hơn.
- **Naive dense retrieval chưa đủ cho miền pháp luật:** Naive dense retrieval đạt $\text{Recall@5} = 0.3125$ và $\text{NDCG@10} = 0.2841$, thấp nhất trong năm chiến lược. Điều này cho thấy dense search đơn thuần chưa khai thác tốt các tín hiệu quan trọng như từ khóa, số hiệu văn bản và quan hệ pháp lý.
- **Chất lượng cao đi kèm độ trễ lớn hơn:** Agentic đạt khoảng ~ 413 ms và Reranker đạt khoảng ~ 390 ms, cao hơn Hybrid ở mức ~ 142 ms và Naive ở mức ~ 87 ms. Vì vậy, Agentic phù hợp khi ưu tiên chất lượng câu trả lời, còn Hybrid hoặc Reranker phù hợp hơn khi cần cân bằng giữa chất lượng và tốc độ.



Hình 9: So sánh Recall@5 và NDCG@10 trên năm chiến lược retrieval.

Quan sát Hình 9, ta thấy Agentic đạt điểm cao nhất ở cả Recall@5 và NDCG@10. Tuy nhiên, khoảng cách chất lượng này đi kèm với độ trễ cao hơn, nên lựa chọn chiến lược retrieval cần cân bằng giữa độ chính xác và chi phí suy luận.

Kết luận: Agentic RAG cho chất lượng retrieval tốt nhất trên tập câu hỏi pháp luật, nhờ khả năng tự điều phối chiến lược theo ngữ cảnh từng câu hỏi. Reranker là lựa chọn tối ưu khi cần cân bằng giữa chất lượng và tốc độ. Tuy nhiên, bộ gold set 16 câu hỏi còn nhỏ nên kết quả mang tính tham khảo. Các hướng cải thiện tiềm năng bao gồm mở rộng gold set, tích hợp query rewriting vào Hybrid và tinh chỉnh số hop cho Graph Retrieval theo từng loại câu hỏi.

VI. Tài liệu tham khảo

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks”, in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [2] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models”, in *International Conference on Learning Representations (ICLR)*, 2023.
- [3] D. Edge et al., “From local to global: A graph RAG approach to query-focused summarization”, *arXiv preprint arXiv:2404.16130*, 2024.
- [4] S.-Q. Yan et al., “Corrective retrieval augmented generation”, *arXiv preprint arXiv:2401.15884*, 2024.
- [5] A. Asai et al., “Self-RAG: Learning to retrieve, generate, and critique through self-reflection”, in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] S. Es et al., “RAGAS: Automated evaluation of retrieval augmented generation”, *arXiv preprint arXiv:2309.15217*, 2023.
- [7] S. Robertson et al., “The probabilistic relevance framework: BM25 and beyond”, *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [8] G. V. Cormack et al., “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods”, in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [9] J. Chen et al., “BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation”, *arXiv preprint arXiv:2309.07597*, 2024.
- [10] LangChain AI, *LangGraph: Building stateful, multi-actor applications with LLMs*, 2024.
- [11] A. A. Hagberg et al., *Exploring network structure, dynamics, and function using NetworkX*, 2008.

Bảng thuật ngữ

Thuật ngữ	Tạm dịch	Mô tả
<i>Agentic RAG</i>	RAG tác tử	Hướng mở rộng của RAG trong đó LLM đóng vai trò agent: tự lựa chọn chiến lược retrieval, tự đánh giá chất lượng ngữ cảnh và lặp lại nếu cần, trước khi sinh câu trả lời cuối. Khác với RAG truyền thống chạy theo pipeline cố định, Agentic RAG xử lý được các câu hỏi đòi hỏi nhiều bước suy luận.
<i>Deep Agents</i>	—	Thư viện xây dựng AI agent theo hướng khai báo đơn giản: lập trình viên chỉ cần truyền vào model, danh sách tool và system prompt, thư viện tự quản lý toàn bộ vòng lặp Reason–Act–Observe bên trong. Deep Agents tích hợp sẵn các tool phổ biến như thao tác file, thực thi shell và gọi subagent.
<i>Graph Retrieval</i>	Truy xuất qua đồ thị	Chiến lược retrieval tổ chức tập tài liệu thành đồ thị có hướng, trong đó mỗi edge biểu diễn một quan hệ như tham chiếu, sửa đổi hay thay thế. Quá trình truy xuất bắt đầu từ seed documents tìm được qua dense search, sau đó duyệt theo các edge để thu thập thêm tài liệu liên quan qua nhiều hop.
<i>Hop</i>	Bước duyệt đồ thị	Một bước nhảy từ node hiện tại sang node láng giềng theo chiều edge trong Graph Retrieval. Số hop kiểm soát độ sâu traversal: nhiều hop hơn giúp thu thêm thông tin gián tiếp nhưng kéo theo nhiều tài liệu ít liên quan, làm loãng ngữ cảnh đưa vào LLM.
<i>LangGraph</i>	—	Thư viện mã nguồn mở do LangChain phát triển, cho phép mô hình hóa luồng xử lý LLM dưới dạng đồ thị có trạng thái (stateful graph). Mỗi node thực hiện một đơn vị công việc, conditional edge cho phép rẽ nhánh động dựa trên kết quả trung gian, và checkpointing lưu trạng thái giữa các lượt hội thoại.

Thuật ngữ	Tạm dịch	Mô tả
<i>NDCG@k</i>	Normalized Discounted Cumulative Gain tại k	Metric đánh giá retrieval đo đồng thời mức độ bao phủ và chất lượng xếp hạng: tài liệu liên quan được xếp càng cao thì đóng góp điểm càng lớn, và điểm bị giảm dần (discount) theo thứ hạng. Điểm cuối được chuẩn hóa về khoảng $[0, 1]$ so với thứ tự xếp hạng lý tưởng. NDCG phạt chiến lược tìm đúng tài liệu nhưng xếp sai vị trí, điều mà Recall hoàn toàn bỏ qua.
<i>Reranker</i>	Mô hình xếp hạng lại	Mô hình CrossEncoder đánh giá lại độ liên quan giữa câu hỏi và từng đoạn văn bản sau retrieval, giữ lại các chunk có điểm cao nhất trước khi đưa vào LLM. Khác với bi-encoder dùng trong dense search, CrossEncoder xử lý câu hỏi và ngữ cảnh cùng lúc nên chính xác hơn nhưng tốn thời gian hơn.
<i>RRF</i>	Reciprocal Rank Fusion	Phương pháp hợp nhất kết quả từ nhiều chiến lược retrieval bằng cách tính điểm nghịch đảo thứ hạng cho mỗi tài liệu rồi cộng tổng. RRF loại bỏ sự chênh lệch thang điểm giữa BM25 và dense search mà không cần hiệu chỉnh tham số phức tạp.
<i>Seed retrieval</i>	Truy xuất khởi điểm	Bước đầu tiên trong Graph Retrieval, dùng dense search để tìm một tập tài liệu ban đầu có độ tương đồng ngữ nghĩa cao nhất với câu hỏi. Các tài liệu này đóng vai trò điểm khởi đầu để duyệt đồ thị quan hệ qua các hop tiếp theo, do đó chất lượng seed ảnh hưởng trực tiếp đến phạm vi thông tin được thu thập.

Phụ lục

1. **Resources:** Các file code gợi ý (hint) và bài giải (solution) có thể được tải tại [đây](#). (Lưu ý: Sau khi hết deadline của phần bài này, ad mới copy các nội dung bài giải vào đường dẫn).

2. **Rubric:**

Mục	Kiến Thức	Đánh Giá
II.	- Giới hạn của RAG pipeline tĩnh và động lực ra đời của Agentic RAG. - LangGraph: State, Node, Edge, Conditional Edge và Checkpointer. - Deep Agents: bộ công cụ tích hợp sẵn và vòng lặp Reason-Act-Observe tự động. - Graph Retrieval: khái niệm Node, Edge, Hop và đánh đổi khi tăng số hop.	- Giải thích được khi nào dùng LangGraph và khi nào dùng Deep Agents. - Mô tả được luồng Reason-Act-Observe trong một hệ thống Agentic RAG. - Phân biệt được Graph Retrieval với dense và hybrid retrieval. - Nhận diện được pattern phù hợp cho từng loại bài toán RAG.
III. & IV.	- Kiến trúc bốn lớp của project <code>aio-agentic-rag</code>: Ingestion, Retrieval, Agent/Tools và Serving. - Năm chiến lược retrieval và trường hợp sử dụng của từng chiến lược. - Cách đóng gói retrieval function thành LangChain tool bằng <code>@tool</code>. - Thiết kế hai tầng agent: Legal RAG agent và orchestrator. - Separator pháp luật trong chunker và graph traversal bằng <code>out_edges/in_edges</code>.	- Cài đặt và chạy được hệ thống theo đúng thứ tự bốn bước. - Giải thích được tại sao docstring của tool ảnh hưởng đến quyết định của agent. - Hiểu được vai trò của graph index trong câu hỏi multi-hop pháp luật. - Chạy được benchmark và đọc được kết quả Recall@5 và NDCG@10.

3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,6K thành viên



Hình 10: Hình ảnh group facebook AIO Q&A.